Lecture 6

- degree of belief P in a hypothesis h (Bayesian)
  - $P(h \mid S) \leftarrow P(h)$
  - $P(h \mid S)$: Posterior
  - $P(S \mid h)$: Likelihood
- (h: meaning) learning representations h from data S
- (h: function) $h : \mathcal{X} \to \mathcal{Y}$
- $[\arg\max_h \text{Lik}(h) = \arg\min_h \text{NLL}(h) = \arg\min_h \text{KL-div}(h)]$ - Gaussian - $\arg\min_h \text{MSE}(h)$ (Least Squares)
  - Gaussian noise - CLT
- ERM: $\arg\max_h L_S(h)$

Preliminaries
- Tools
- Linear Algebra
- Matrix Calculus
- Probability

Introduction
- Neural Networks
- Optimization
- Neural Networks + Optimization
- Generalization
- etc.

Inductive Bias - Restricted Hypothesis Space

T: to find a minimizer over

- the whole function space $\mathcal{F}(\mathcal{X}, \mathcal{Y}) := \{f : \mathcal{X} \to \mathcal{Y}\}$
- a (restricted) hypothesis space $\mathcal{H} \subset \mathcal{F}(\mathcal{X}, \mathcal{Y})$

e.g., $(\mathcal{F}(\mathcal{X}, \mathcal{Y}) \to \mathcal{H})$

- function space $\to$ parameter space $\{f_\theta : \theta \in \mathbb{R}^p\}$ (NN)
- MLP $\to$ ConvNet
- ...

cf. we have a prior knowledge.

Parameterized Models - Linear Models

Linear Models

- $f(x; w) = \mathbf{1}(w^\top x \geq 0)$[48]: perceptron
- $p(y \mid x; w) = Ber(y \mid \sigma(w^\top x)$[49]): (binary) logistic regression
- $p(y \mid x; W) = Cat(y \mid \text{softmax}(Wx))$: multi-class ...
- $p(y \mid x; w) = \mathcal{N}(y \mid w^\top x, \sigma^2)$: linear regression

linear model $\rightarrow$ feature transformation (nonlinear basis function) $\rightarrow$ parameterized feature extractor $\rightarrow$ going deeper $\rightarrow$ inductive bias $\rightarrow$ less inductive bias (?)[50]

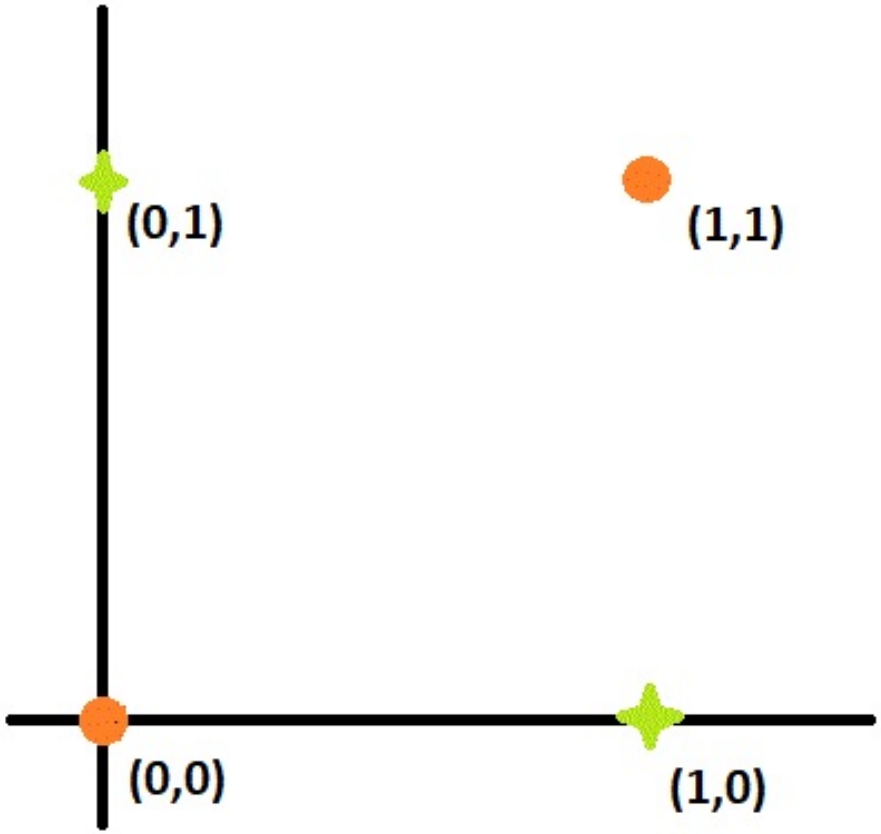
[48] also called the Heaviside step function

[49] logistic function $\sigma : \mathbb{R} \rightarrow (0, 1)$ with $\sigma(x) = \frac{e^x}{1+e^x}$

[50] perceptron $\rightarrow$ perceptron + nonlinear basis function $\rightarrow$ 2-layer perceptron $\rightarrow$ MLP $\rightarrow$ ConvNet $\rightarrow$ Transformer

XOR

For the XOR problem, it is clear that the corresponding data is not linearly separable, so a perceptron cannot represent the XOR mapping [MP69].



(Nonlinear) Basis Functions

For this reason, it is important to use nonlinear activation functions.[51][52]

$$f(x;\theta) = W\phi(x) + b$$

where ϕ is a (nonlinear) basis function.

[51] A composite of linear functions reduces to a linear function (not enough expressive power - cf. XOR).

[52] e.g., $\phi(x) = [x_1, x_2, x_1^2, x_2^2, x_1 x_2]^\top$ or $\phi(x) = |x_1 - x_2|$

Parameterized Feature Extractor

Having to specify the feature transformation by hand ($\phi(x)$) is very limiting.

$$f(x; \theta, \theta') = W\phi(x; \theta') + b$$

where $\phi(\cdot, \theta')$ is a learnable feature extractor.

Activation Functions

$$\text{logistic}(x) = \frac{e^x}{1+e^x} \in (0,1) \qquad \text{(a/the sigmoid)}$$

$$\text{softmax}(x) = \frac{e^x}{\sum_j e^{x_j}}$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \in (-1,1)$$

$$\text{softplus}(x) = \log(1+\exp(x)) > 0$$

$$\text{ReLU}(x) = \max(0,x)$$

$$\text{LeakyReLU}(x) = \max(ax,x) \qquad (a = 0.01)$$

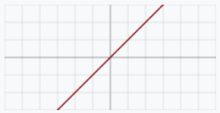
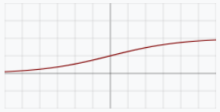
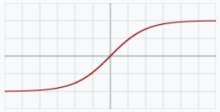
$$\text{PReLU}(x) = \max(ax,x) \qquad (a_0 = 0.25;\ a:\ \text{learnable})$$

$$\text{ELU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ a(e^x - 1) & \text{if } x < 0 \end{cases} \qquad (a = 1)$$

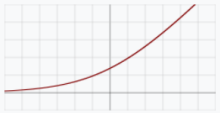
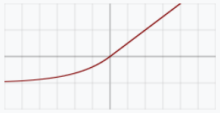
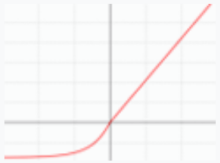
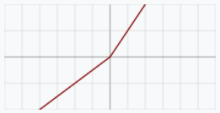
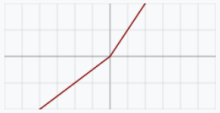
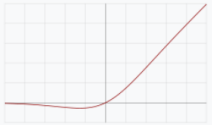
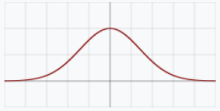
$$\text{SiLU}(x) = x\,\text{logistic}(x) \qquad (= \text{Swish-1})$$

$$\text{GELU}(x) = xP(X \leq x) \approx x\,\text{logistic}(1.702x) \qquad (X \sim \mathcal{N}(0,1))$$

Activation Functions

Name	Plot	Function, $g(x)$	Derivative of g, $g'(x)$	Range	Order of continuity
Identity		x	1	$(-\infty, \infty)$	C^∞
Binary step		$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	0	$\{0, 1\}$	C^{-1}
Logistic, sigmoid, or soft step		$\sigma(x) \doteq \frac{1}{1+e^{-x}}$	$g(x)(1-g(x))$	$(0, 1)$	C^∞
Hyperbolic tangent (tanh)		$\tanh(x) \doteq \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$1 - g(x)^2$	$(-1, 1)$	C^∞
Soboleva modified hyperbolic tangent (smht)		$\mathrm{smht}(x) \doteq \frac{e^{ax} - e^{-bx}}{e^{cx} + e^{-dx}}$		$(-1, 1)$	C^∞
Rectified linear unit (ReLU)[8]		$(x)^+ \doteq \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases} = \max(0, x) = x\mathbf{1}_{x>0}$	$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$	$[0, \infty)$	C^0

Activation Functions

Gaussian Error Linear Unit (GELU)[2]		$\frac{1}{2}x\left(1+\operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)\right)$ $= x\Phi(x)$	$\Phi(x)+x\phi(x)$	$(-0.17\ldots,\infty)$	C^∞
Softplus[9]		$\ln(1+e^x)$	$\frac{1}{1+e^{-x}}$	$(0,\infty)$	C^∞
Exponential linear unit (ELU)[10]		$\begin{cases}\alpha\left(e^x-1\right) & \text{if } x\le 0\\ x & \text{if } x>0\end{cases}$ with parameter α	$\begin{cases}\alpha e^x & \text{if } x<0\\ 1 & \text{if } x>0\end{cases}$	$(-\alpha,\infty)$	$\begin{cases}C^1 & \text{if } \alpha=1\\ C^0 & \text{otherwise}\end{cases}$
Scaled exponential linear unit (SELU)[11]		$\lambda\begin{cases}\alpha(e^x-1) & \text{if } x<0\\ x & \text{if } x\ge 0\end{cases}$ with parameters $\lambda=1.0507$ and $\alpha=1.67326$	$\lambda\begin{cases}\alpha e^x & \text{if } x<0\\ 1 & \text{if } x\ge 0\end{cases}$	$(-\lambda\alpha,\infty)$	C^0
Leaky rectified linear unit (Leaky ReLU)[12]		$\begin{cases}0.01x & \text{if } x\le 0\\ x & \text{if } x>0\end{cases}$	$\begin{cases}0.01 & \text{if } x<0\\ 1 & \text{if } x>0\end{cases}$	$(-\infty,\infty)$	C^0
Parametric rectified linear unit (PReLU)[13]		$\begin{cases}\alpha x & \text{if } x<0\\ x & \text{if } x\ge 0\end{cases}$ with parameter α	$\begin{cases}\alpha & \text{if } x<0\\ 1 & \text{if } x\ge 0\end{cases}$	$(-\infty,\infty)$	C^0
Sigmoid linear unit (SiLU,[2] Sigmoid shrinkage,[14] SiL,[15] or Swish-1[16])		$\frac{x}{1+e^{-x}}$	$\frac{1+e^{-x}+xe^{-x}}{\left(1+e^{-x}\right)^2}$	$[-0.278\ldots,\infty)$	C^∞
Gaussian		e^{-x^2}	$-2xe^{-x^2}$	$(0,1]$	C^∞

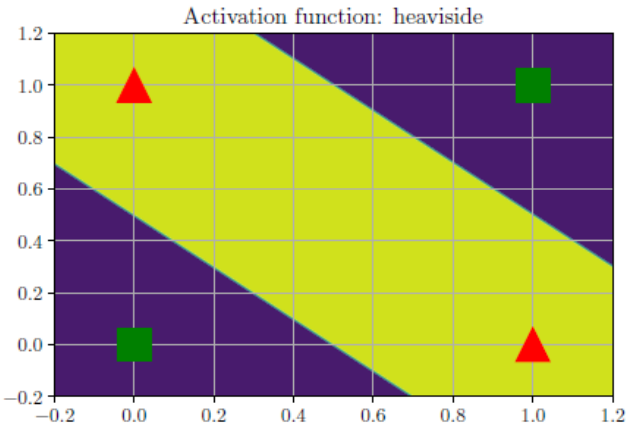
Activation Functions

- vanishing gradient problem: e.g., a sigmoid function saturates at 1 for large positive inputs, and at 0 for large negative inputs. In these regimes, the gradients of the output wrt the input will be close to zero, so any gradient signal from higher layers will not be able to propagate back to earlier layers. (sigmoid $\rightarrow$ ReLU)
- dead ReLU problem: if the weights are initialized to be large and negative, then it becomes very easy for the ReLU output to go to 0. This will cause the gradient for the weights to go to 0. The algorithm will never be able to escape this situation, so the hidden units will stay permanently off. (ReLU $\rightarrow$ LeakyReLU, PReLU, ELU, SELU)

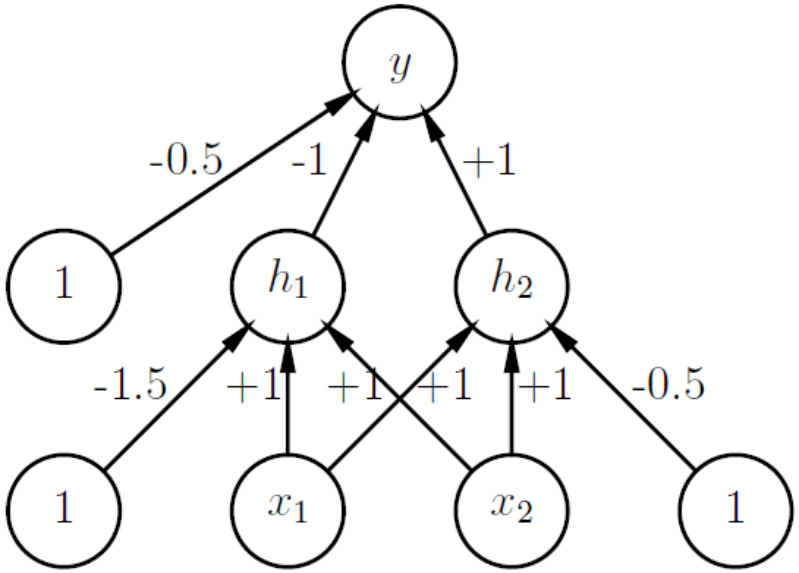
2-layer NNs (one hidden layer)

$$
\begin{aligned}
x &\in \mathbb{R}^{d_0} \\
y &\in \mathbb{R} \\
W_0 x &\in \mathbb{R}^{d_1} \\
\phi(x;\theta') := \sigma(W_0 x) &\in \mathbb{R}^{d_1} \\
w^\top \sigma(W_0 x) &\in \mathbb{R} \\
f(x;\theta) := \mathbf{1}(w^\top \sigma(W_0 x) \geq 0) &\in \{0,1\}
\end{aligned}
$$

A 2-layer NN Can Solve the XOR Problem



(a)



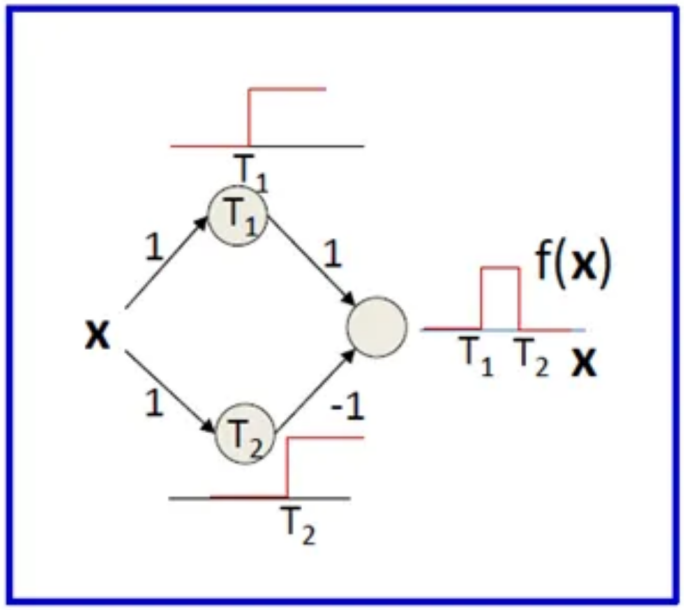
(b)

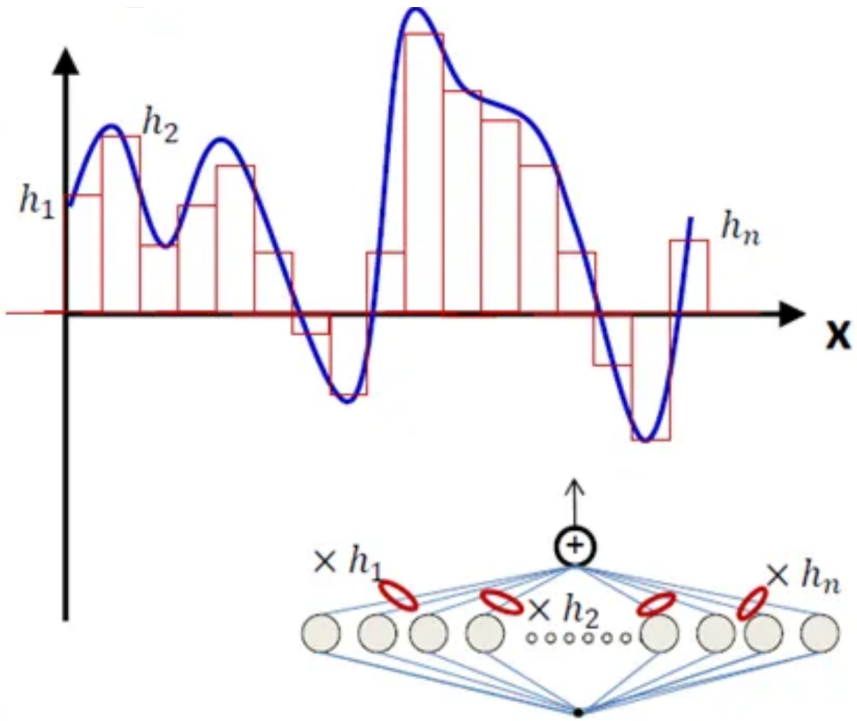
Figure: (b): $h_1 = \text{AND}, h_2 = \text{OR}$

Theorem (Universal Approximation Theorem (informal))

A 2-layer NN with arbitrary width can approximate any target function.

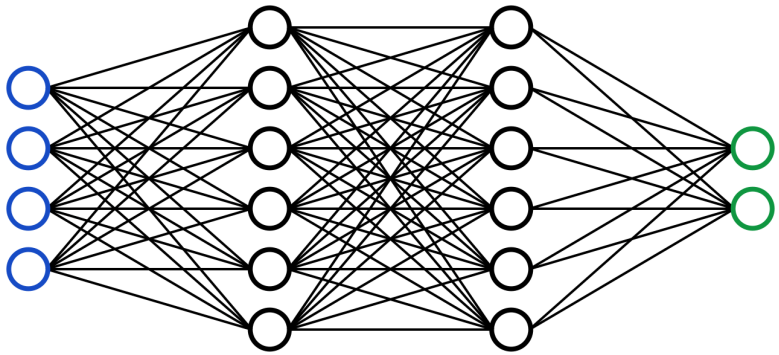
Visual Proof of Universal Approximation Theorem





(Why?)

Multi-Layer Perceptron (MLP)

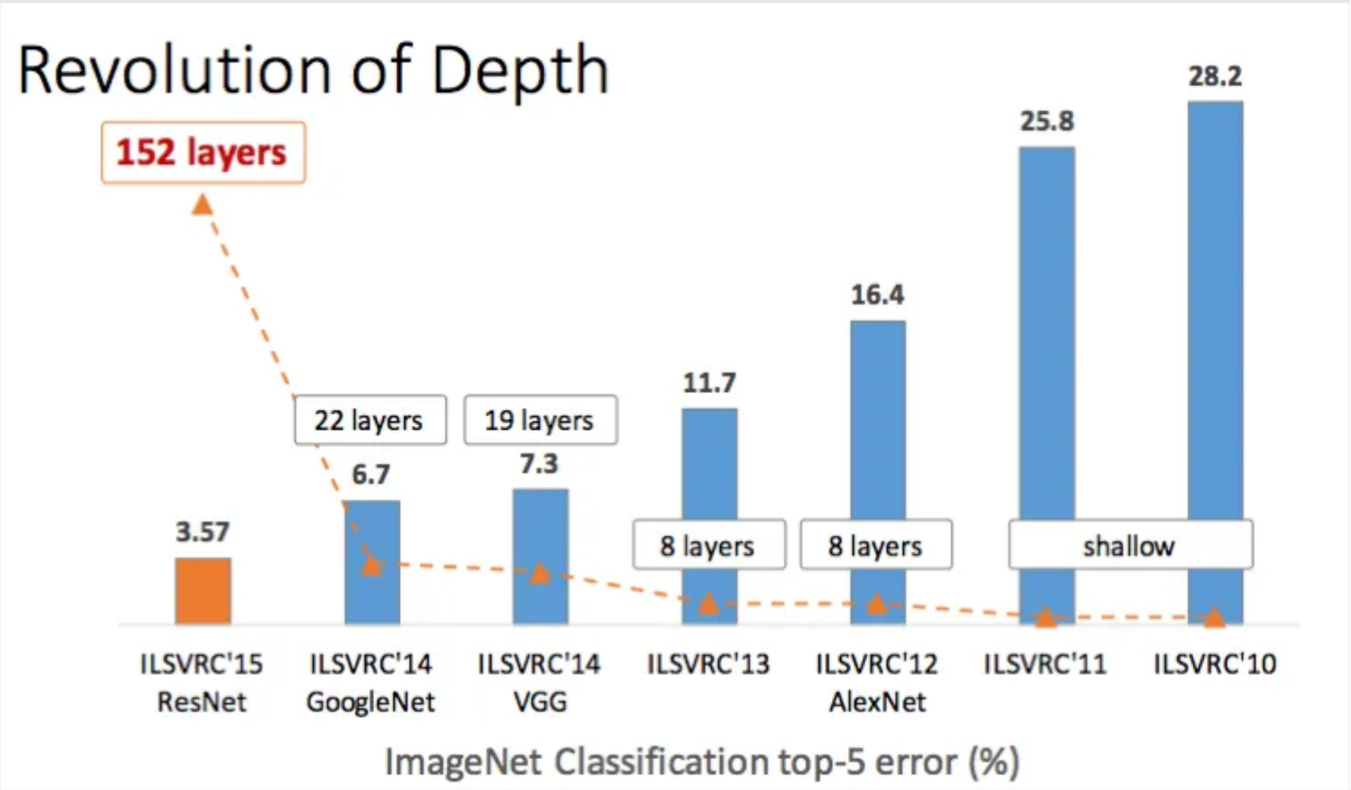


$$
\begin{aligned}
x = x_0 &\in \mathbb{R}^{d_0} \\
x_1 &= \sigma(W_0 x_0) \in \mathbb{R}^{d_1} \\
&\vdots \\
x_{k+1} &= \sigma(W_k x_k) \in \mathbb{R}^{d_{k+1}} \\
&\vdots \\
z &= W_{L-1} x_{L-1} \in \mathbb{R}^{d_L}
\end{aligned}
$$

Why Non-linear Activations?

$$z = \underbrace{W_{L-1} W_{L-2} \cdots W_0}_{W'} x_0 = W' x$$

Going Deeper (Layers)[53]



An MLP with one hidden layer is a universal function approximator. However, various arguments have shown that **DNNs work better than shallow ones**. One possible reason is that we can learn a function in a **compositional or hierarchical** way. (No one knows.)

[53] ResNet outperforms humans ($\sim$ 5% by Andrej Karpathy).

No Free Lunch Theorem

> *All models are wrong, but some models are useful.*
> *— George Box [BD87]*

There is no single best model that works optimally for all kinds of problems.

Inductive Bias

Some prior knowledge about data

- NNs for Images
- NNs for Sequences
- NNs for Graphs
- NNs for Sets
- NNs for 3D Point Clouds
- NNs for Molecules

inductive bias: *the set of assumptions (prior) that the model makes in order to generalize to new inputs*

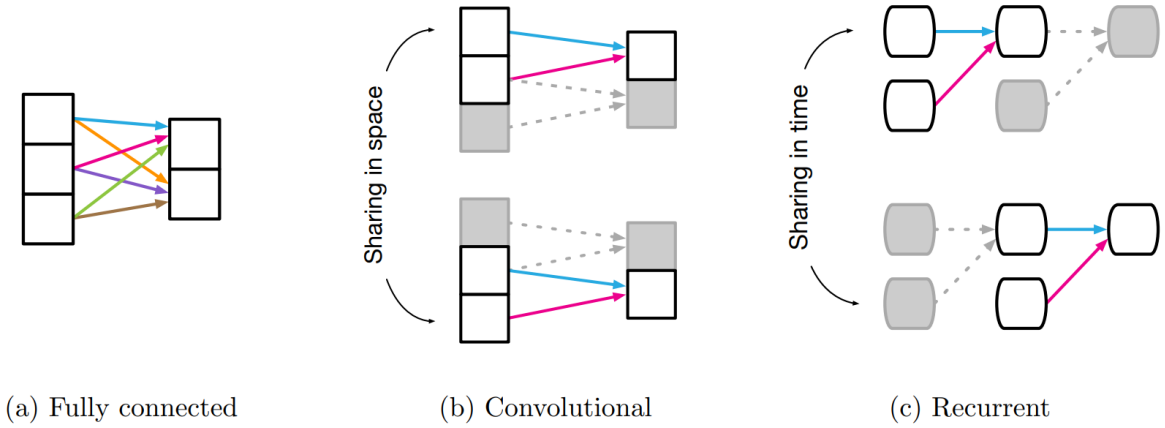


Figure: [BHB+18]

- locality, stationarity (Conv)
- multi-scale (from simple to abstract in a hierarchical way)
- permutation invariance (GNN, Deepset)
- nearest neighbors (kNN), maximal margin (SVM) $\cdots$

linear model $\rightarrow$ feature transformation (nonlinear basis function) $\rightarrow$ parameterized feature extractor $\rightarrow$ going deeper $\rightarrow$ inductive bias $\rightarrow$ less inductive bias (?)

Loss

linear regression - least squares

$$S = (X, y) \qquad (X \in \mathbb{R}^{n \times p}, y \in \mathbb{R}^n)$$

$$\begin{aligned} L_S(w) &= \frac{1}{2}\|Xw - y\|^2 \\ &= \frac{1}{2}(Xw - y)^\top (Xw - y) \\ &= \frac{1}{2}\left(w^\top X^\top X w - 2y^\top X w + y^\top y\right) \end{aligned}$$

- For a linear regression, the loss is quadratic, and thus *convex*. Therefore, it is easy to find a minimum.
- However, for a DNN, the loss is generally *nonconvex*. So we need to use gradient-based optimization (e.g., GD) to find a minimum.